

UNIT-II

White-Box Testing: need, Logic Coverage criteria, Basis Path testing, Graph matrices, Loop testing, data flow testing, mutation testing.

2.1 White-Box Testing

- White box testing (WBT) is also called Structural or Glass box testing.
- White box testing involves looking at the structure of the code.
- In Black box testing can't we write the code and we simply test the software
- The supporting reasons for white-box testing are given below:
 - White-box testing techniques are used for testing the module for initial stage testing. Black-box testing is the second stage for testing the software. Though test cases for black box can be designed earlier than white-box test cases, they cannot be executed until the code is produced and checked using white-box testing techniques.
 - White-box testing is complementary to black-box testing, bugs which can be revealed by white-box testing, but not through black-box testing. Some portions in the code which are not checked when executing functional test cases, but these will be executed and tested by white-box testing.
 - Errors which have come from the design phase will also be reflected in the code, therefore we must execute white-box test cases for verification of code.
 - Logical path is not likely to be executed White-box testing explores these paths too.
 - Typographical errors are not observed and go undetected and are not covered by black-box testing techniques. White-box testing techniques help detect these errors.

2.2 Logic Coverage Criteria

- Structural testing considers the program code, and test cases are designed based on the logic of the program such that every element of the logic is covered.
- The following are the basic forms of logic coverage.

Statement Coverage

- logic coverage can be identified in the form of statements
- It is assumed that if all the statements of the module are executed once, every bug will be notified.
- Consider the following code segment

```
scanf ("%d", &x);
scanf ("%d", &y);
while (x != y)
{
    if (x > y)
        x = x - y;
    else
        y = y - x;
}
printf ("x = ", x);
printf ("y = ", y);
```

- If we want to cover every statement in the above code, then the following test cases must be designed:

Test case 1: $x = y = n$, where n is any number

Test case 2: $x = n$, $y = n^!$, where n and $n^!$ are different numbers.

- Test case 1 just skips the while loop and all loop statements are not executed.
- So considering test case 2, the loop is also executed. So two more cases are designed:

Test case 3: $x > y$

Test case 4: $x < y$

- These test cases will cover every statement in the code segment.
- Statement coverage is a poor criteria for logic coverage test case 3 and 4 are sufficient to execute all the statements in the code. But, if we execute only test case 3 and 4, then conditions and paths in test case 1 will never be tested and errors will go undetected.
- Thus, statement coverage is a necessary but not a sufficient criteria for logic coverage.

Decision or Branch Coverage

- Branch coverage states that each decision takes on all possible outcomes (True or False) at least once.
- In the above sample code **while** and **if** statements have two outcomes: True and False.
- So test cases must be designed such that both outcomes for **while** and **if** statements are tested. The test cases are:

Test case 1: $x = y$

Test case 2: $x \neq y$

Test case 3: $x < y$

Test case 4: $x > y$

Condition Coverage

- It states that each condition in a decision takes on all possible outcomes at least once.
- For example consider the following statement:

`while ((I<5) && (J<COUNT))`

The different test cases are:

Test case 1: $I \leq 5$, $J < \text{COUNT}$

Test case 2: $I > 5$, $J > \text{COUNT}$

Decision/condition Coverage

- Condition coverage in a decision does not mean that the decision has been covered.
- If the decision **If (A && B)** is being tested, the condition coverage would allow one to write two test cases:

Test case 1: A is True, B is False.

Test case 2: A is False, B is True.

- These test cases would not cause the THEN clause of the IF to execute (i.e. execution of decision).

Multiple condition coverage

- In multiple conditions taken all combinations of different multiple conditions.

- For example, if an AND condition is False, none of the subsequent conditions in the expression will be evaluated. Similarly, if an OR condition is True, none of the subsequent conditions will be evaluated.
- Thus, condition coverage and decision/condition coverage need not necessarily uncover all the errors.
- Therefore, multiple condition coverage write sufficient test cases such that all possible combinations of condition outcomes in each decision are invoked at least once.

- For the statement **If (A && B)** the test cases would be:

Test case 1: $A = \text{True}, B = \text{True}$

Test case 2: $A = \text{True}, B = \text{False}$

Test case 3: $A = \text{False}, B = \text{True}$

Test case 4: $A = \text{False}, B = \text{False}$

2.3 Basis Path Testing

- Basis Path Testing (BPT) is the oldest structural testing technique.
- The technique is based on the control structure of the program.
- Based on the control structure, a flow graph is prepared and all the possible paths can be covered and executed during testing.
- The guidelines for effective path testing are:
 - Path Testing is based on control structure of the program for which flow graph is prepared.
 - It requires complete knowledge of the program's structure.
 - It is closer to the developer and used by him to test his module.
 - The effectiveness of path testing is reduced with the increase in size of software under test.
 - Choose enough paths in a program such that maximum logic coverage is achieved.

2.3.1 Control Flow Graph

- The control flow graph is a graphical representation of control structure of a program.
- Flow graph can be prepared as a directed graph.
- A Directed graph (V,E) consists of set of vertices V and edges E. Flow graph notations are:

Node

- It represents one or more procedural statements.
- The nodes are denoted by a circle.
- These are numbered or labeled.

Edges or links

- They represent the flow of control in a program.
- This is denoted by an arrow on the edge.
- An edge must terminate at a node.

Decision node

- A node with more than one arrow leaving it is called a decision node.

Junction node

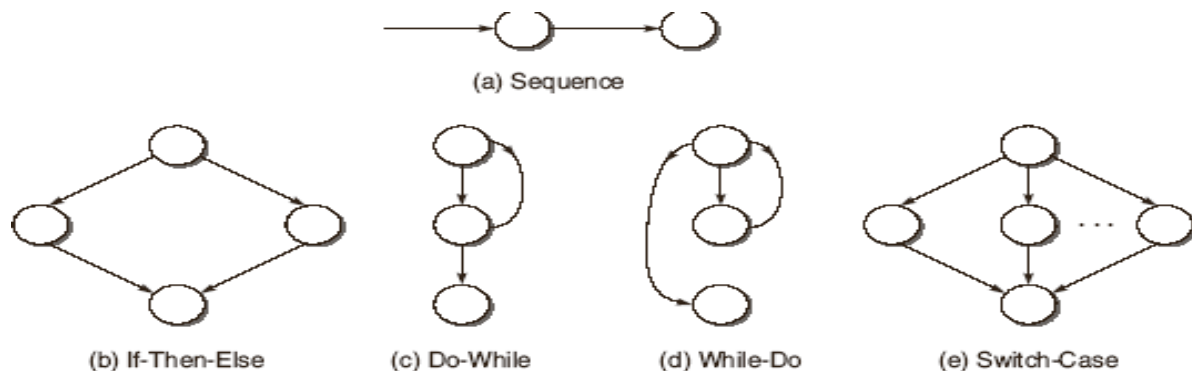
- A node with more than one arrow entering it is called a junction.

Regions

- Areas bounded by edges and nodes are called regions.
- When counting the regions, the area outside the graph is also considered a region.

2.3.2 Flow Graph Notations for Different Programming Constructs

- Flow graph is prepared on the basis of control structure of a program, some fundamental graphical notations are as shown below.
- Sequential statements having no conditions or loops can be merged in a single node.
- So the flow graph is also known as decision-to-decision-graph or DD graph.



2.3.3 Path Testing Terminology

Path:

- A path is a sequence of instructions or statements that starts at an entry, junction, or decision and ends at another or possibly the same junction, decision or exit.

Segment:

- Path consists of segments. The smallest segment is a link, that is, a single process that lies between two nodes.

Path Segment:

- A path segment is a succession of consecutive links that belongs to some path.

Length of a Path:

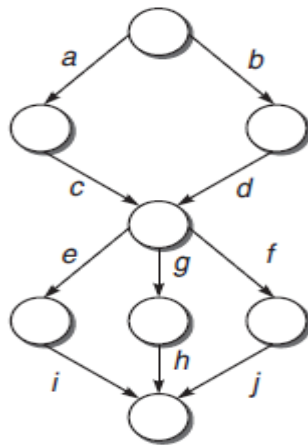
- The length of a path is measured by the number of links in it and not by the number of instructions or statements executed along the path.

Independent Path:

- An independent path is any path through the graph that introduces at least one new set of processing statements or new conditions.

2.3.4 Cyclomatic Complexity

- It measures the logical complexity of a program by considering the number of paths in the control flow graph of the program..
- Even for simple programs, if they contain at least one cycle, the number of paths is infinite.
- Therefore, considers only independent paths.



Sample graph

- In the above graph there are six possible paths: **acei, acgh, acfj, bdei, bdgh, bdfj**. In that only four are independent, as the other two are always a linear combination of the other four paths.
- Therefore, it can be determined that is a strongly connected graph (in which each node can be reached from any other node),
- The number of independent paths is given by

$$V(G) = e - n + 1$$

where n is the number of nodes and e is the number of arcs/edges.

- It may be possible that the graph is not strongly connected, to make the graph strongly connected, we add an arc from the last node to the first node of the graph.
- But by doing this, we increase the number of arcs by 1 and therefore, the number of independent paths is given by

$$V(G) = e - n + 2$$

- This is called the cyclomatic number of a program. We can calculate the cyclomatic number only by knowing the number of decision nodes d in the program. It is given by

$$V(G) = d + 1$$

- The program may contain several procedures. These procedures can be represented as separate flow graphs.
- These procedures can be called from any point but the connections for calling are not shown explicitly.
- The cyclomatic number of the whole graph is then given by the sum of the numbers of each graph.
- The cyclomatic number is given by

$$V(G) = e - n + 2p$$

p is the number of graphs and e and n are referred to as the whole graph.

- Miller's theorem becomes $V(G) = d + p$

Formulae Based on Cyclomatic Complexity

- Based on the cyclomatic complexity, the following formulae are used:

Cyclomatic complexity number can be derived through any of the following three formulae

1. $V(G) = e - n + 2p$
2. $V(G) = d + p$

3. $V(G)$ = number of regions in the graph

Calculating the number of decision nodes for Switch-Case/Multiple If-Else

- When a decision node has exactly two arrows leaving it, then we count it as a single decision node.
- For switch-case and multiple if-else statements have more than two arrows leaving a decision node, and in these cases, the formula to calculate the number of nodes is

$$d = k - 1, \text{ where } k \text{ is the number of arrows leaving the node.}$$

Calculating the cyclomatic complexity number of the program having many connected components

- For example, a program P has three components: X, Y, and Z.
- Then we prepare the flow graph for P and for components, X, Y, and Z.
- The complexity number of the whole program is:

$$V(G) = V(P) + V(X) + V(Y) + V(Z)$$

- The cyclomatic complexity number of the full program by counting the number of nodes and edges in all the components of the program then the formula is

$$V(G) = e - n + 2P$$

- The complexity number derived collectively will be same as calculated as

$$V(P \cup X \cup Y \cup Z) = V(P) + V(X) + V(Y) + V(Z)$$

Guidelines for Basis Path Testing

- Draw the flow graph using the code provided for which we have to write test cases.
- Determine the cyclomatic complexity of the flow graph.
- Cyclomatic complexity provides the number of independent paths.
- Based on every independent path, choose the data such that this path is executed.

Example

```
main()  
{  
    int number, index;  
1.  printf("Enter a number");  
2.  scanf("%d", &number);  
3.  index = 2;  
4.  while(index <= number - 1)  
5.  {  
6.      if (number % index == 0)  
7.      {  
8.          printf("Not a prime number");  
9.          break;  
10.     }  
11.     index++;  
12.     }  
13.     if(index == number)  
14.         printf("Prime number");  
15. } //end main
```

Draw the DD graph for the program

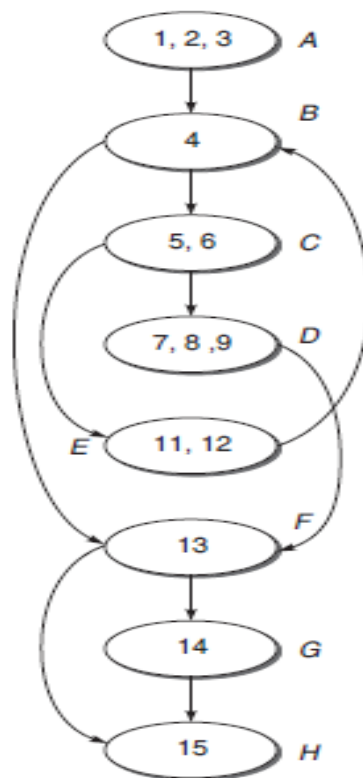
- Calculate the cyclomatic complexity of the program using all the methods.
- List all independent paths.
- Design test cases from independent paths.

Solution

(a) DD graph

For a DD graph, the following actions must be done:

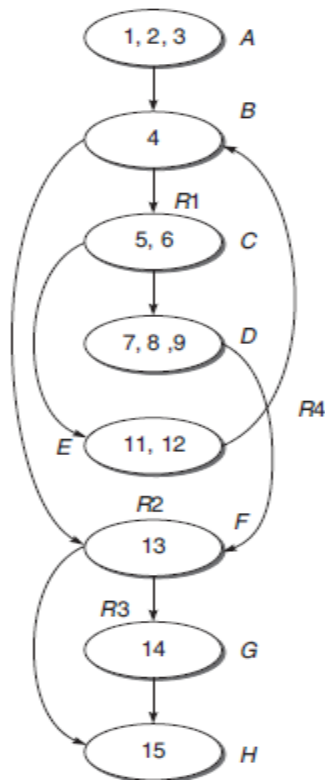
- Put the line numbers on the execution statements of the program.
- Start numbering the statements after declaring the variables.
- Otherwise, start from the statement where a variable has been initialized.
- Put the sequential statements in one node. For example, statement 1, 2, and 3 have been put inside one node.
- Put the edges between the nodes according to their flow of execution.
- Put alphabetical numbering on each node like A, B, etc.



DD graph

(b) Cyclomatic complexity

$$\begin{aligned} \text{(i) } V(G) &= e - n + 2 * p \\ &= 10 - 8 + 2 \\ &= 4 \end{aligned}$$



DD graph showing region

(ii) $V(G) = \text{Number of predicate nodes} + 1$
 $= 3 \text{ (Nodes B, C, and F)} + 1$
 $= 4$

(iii) $V(G) = \text{Number of regions}$
 $= 4(R1, R2, R3, R4)$

(c) Independent paths

- The cyclomatic complexity of the graph is 4, there will be 4 independent paths in the graph they are:
 - A-B-F-H
 - A-B-F-G-H
 - A-B-C-E-B-F-G-H
 - A-B-C-D-F-H

(d) Test case design from the list of independent paths

Test case ID	Input num	Expected result	Independent paths covered by test case
1	1	No output is displayed	A-B-F-H
2	2	Prime number	A-B-F-G-H
3	4	Not a prime number	A-B-C-D-F-H
4	3	Prime number	A-B-C-E-B-F-G-H

Example

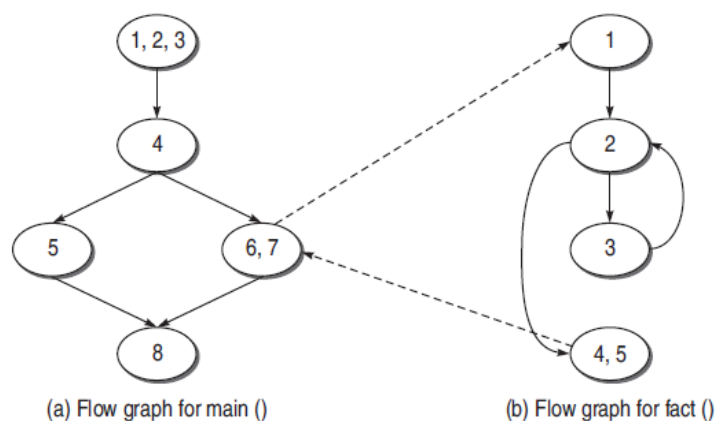
Consider the program for calculating the factorial of a number. It consists of main() program and the module fact(). Calculate the individual cyclomatic complexity number for main() and fact() and then, the cyclomatic complexity for the whole program.

```
main()
{
    int number;
    int fact();
1.   clrscr();
2.   printf("Enter the number whose factorial is to be found out");
3.   scanf("%d", &number);
4.   if(number < 0)
5.       printf("Facorial cannot be defi ned for this number);
6.   else
7.       printf("Factorial is %d", fact(number));
8.   }
int fact(int number)
{
    int index;
1.   int product =1;
2.   for(index=1; index<=number; index++)
3.       product = product * index;
4.   return(product);
5.   }
```

Solution

DD graph

The DD graph of the program is shown below



Cyclomatic complexity of main()

$$\begin{aligned} \text{(a) } V(M) &= e - n + 2p \\ &= 5 - 5 + 2 \\ &= 2 \end{aligned}$$

$$\begin{aligned} \text{(b) } V(M) &= d + p \\ &= 1 + 1 \end{aligned}$$

$$= 2$$

$$(c) V(M) = \text{Number of regions} = 2$$

Cyclomatic complexity of fact()

$$(a) V(R) = e - n + 2p$$

$$= 4 - 4 + 2$$

$$= 2$$

$$(b) V(R) = \text{Number of predicate nodes} + 1$$

$$= 1 + 1$$

$$= 2$$

$$(c) V(R) = \text{Number of regions} = 2$$

Cyclomatic complexity of the whole graph considering the full program

$$(a) V(G) = e - n + 2p$$

$$= 9 - 9 + 2 * 2$$

$$= 4$$

$$= V(M) + V(R)$$

$$(b) V(G) = d + p$$

$$= 2 + 2$$

$$= 4$$

$$= V(M) + V(R)$$

$$(c) V(G) = \text{Number of regions}$$

$$= 4$$

$$= V(M) + V(R)$$

2.3.5 Applications of Path Testing

Path testing has been better suitable as compared to other testing methods some of its applications are:

Thorough testing / More coverage

- Path testing provides us the best code coverage leading to a thorough testing.
- It is considered better as compared to statement or branch coverage methods because the basis path provides us the number of test cases that must be executed for full coverage.

Unit testing

- Path testing is mainly used for testing of a module.
- In unit testing, there are chances of identify errors of decision outcomes or control flow problems which are hidden.

Integration testing

- Modules in a program may call other modules or be called by some other module, there may be chances of interface errors during calling of the modules.

Maintenance testing

- Path testing is also necessary with the modified version of the software.

- Path testing prepared a unit test it should run on the modified software or a selected path testing can be done
- Path testing is still able to detect any security threats on the interface with the called modules.

Testing effort is proportional to complexity of the software

- Cyclomatic complexity number in basis path testing provides the number of tests to be executed on the software based on the complexity of the software.
- Thus, path testing takes care of the complexity of the software and then derives the number of tests to be carried out.

Basis path testing effort is concentrated on error-prone software

- Basis path testing provides us the number of tests to be executed as a measure of software cyclomatic complexity.
- The cyclomatic number signifies that the testing effort is only on the error-prone part of the software, thus minimizing the testing effort.

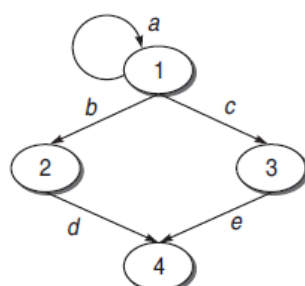
2.4 Graph Matrices

- Flow graph is an effective helps in path testing.
- Path tracing with the use of flow graphs may be a cumbersome and time-consuming activity.
- As the size of graph increases, manual path tracing becomes difficult and leads to errors. A link can be missed or covered twice.
- Develop a software tool which will help in basis path testing is Graph matrix.
- Graph matrix, a data structure, is used for automation of path tracing.

2.4.1 Graph Matrix

- A graph matrix is a square matrix whose rows and columns are equal to the number of nodes in the flow graph.
- Each row and column identifies a particular node and matrix entries represent a connection between the nodes.
- The following points describe a graph matrix:
 - Each cell in the matrix can be a direct connection or link between one node to another node.
 - If there is a connection from node 'a' to node 'b', then it does not mean that there is connection from node 'b' to node 'a'.
 - To represent a graph matrix, digits are used for nodes and letter symbols for edges or connections.

Example 1



Consider the above graph and represent it in the form of a graph matrix.

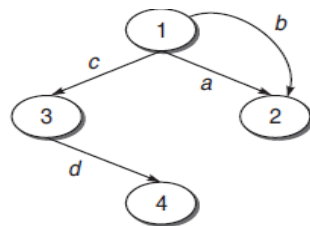
Solution

The graph matrix is shown below.

	1	2	3	4
1	a	b	c	
2				d
3				e
4				

Example 1

Consider the graph and represent it in the form of a graph matrix.



Solution

The graph matrix is shown below.

	1	2	3	4
1		a+b	c	
2				
3				d
4				

2.4.2 Connection Matrix

- Matrix is just a tabular representation and does not provide any useful information.
- If we add link weights to each cell entry, then graph matrix can be used as a powerful tool in testing.
- The links between two nodes are assigned a link weight which becomes the entry in the cell of matrix.
- The link weight provides information about control flow.
- When the connection exists, then the link weight is 1, otherwise 0 (But 0 is not entered in the cell entry of matrix to reduce the complexity).
- A matrix defined with link weights is called a connection matrix.
- The connection matrix for above Examples

	1	2	3	4
1	1	1	1	
2				1
3				1
4				

Example 1

	1	2	3	4
1		1	1	
2				
3				1
4				

Example 2

2.4.3 Use of Connection Matrix in Finding Cyclomatic Complexity Number

- Connection matrix is used to see the control flow of the program. It is also used to find the cyclomatic complexity number of the flow graph.
- The procedure to find the cyclomatic number from the connection matrix:

Step 1: For each row, count the number of 1s and write it in front of that row.

Step 2: Subtract 1 from that count. Ignore the blank rows, if any.

Step 3: Add the final count of each row.

Step 4: Add 1 to the sum calculated in Step 3.

Step 5: The final sum in Step 4 is the cyclomatic number of the graph.

Example

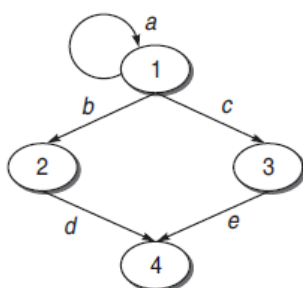
	1	2	3	4	
1	1	1	1		3 - 1 = 2
2				1	1 - 1 = 0
3				1	1 - 1 = 0
4					
Cyclomatic number = 2 + 1 = 3					

2.4.4 Use of Graph Matrix for Finding Set of All Paths

- The purpose of developing graph matrices is to produce a set of all paths between all nodes.
- It is use for path tracing to find k -link paths from one node.
- For example, how many 2-link paths are there from one node to another node? This process is done for every node resulting in the set of all paths.
- This set can be obtained with the help of matrix operations.
- The power operation on matrix expresses the relation between each pair of nodes via intermediate nodes.
- The square of matrix represents path segments that are 2-links long. Similarly, the cube power of matrix represents path segments that are 3-links long.

Example

Consider the graph matrix for the below graph and find 2-link paths for each node.



	1	2	3	4
1	a	b	c	
2				d
3				e
4				

Solution

- For finding 2-link paths, we should square the matrix.
- Squaring the matrix yields a new matrix having 2-link paths.

$$\begin{pmatrix} a & b & c & 0 \\ 0 & 0 & 0 & d \\ 0 & 0 & 0 & e \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} a & b & c & 0 \\ 0 & 0 & 0 & d \\ 0 & 0 & 0 & e \\ 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} a^2 & ab & ac & bd + ce \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

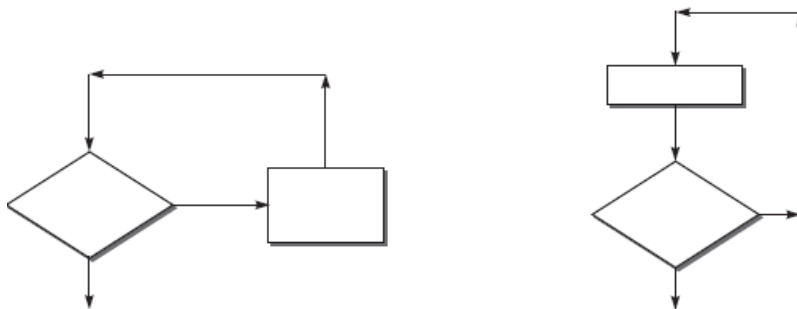
- The resulting matrix shows all the 2-link paths from one node to another.
- For example, from node 1 to node 2, there is one 2-link, i.e., ab .

2.5 Loop Testing

- Loop testing can be viewed as an extension to branch coverage.
- If loops are not tested properly, bugs can go undetected.
- Loop testing can be done effectively while performing unit testing by the developer.
- Sufficient test cases should be designed to test every loop thoroughly.
- There are four different kinds of loops.

Simple loops

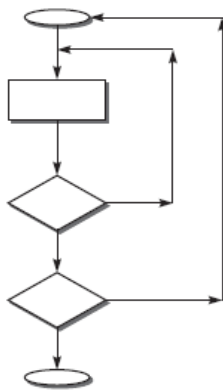
Simple loops mean, we have a single loop in the flow.



- The following test cases should be considered for simple loops while testing them:
 - Check whether you can bypass the loop or not.
 - Check whether the loop control variable is negative.
 - Write one test case that executes the statements inside the loop.
 - Write test cases for a typical number of iterations through the loop.
 - Write test cases for checking the boundary values of the maximum and minimum number of iterations in the loop.
 - It means we should test for min, min+1, min-1, max-1, max, and max+1 number of iterations through the loop.

Nested loops

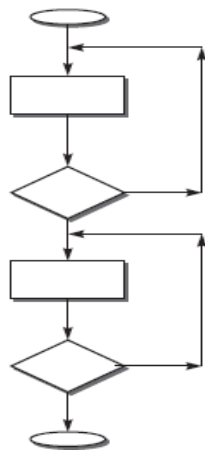
- When two or more loops are embedded, it is called a nested loop.
- If we have nested loops in the program, it becomes difficult to test.
- So we use simple tests to test the nested loops, then the number of possible test cases grows.
- The strategy is to start with the innermost loops while holding outer loops to their minimum values. Continue this outward in this manner until all loops have been covered.



Nested loops

Concatenated loops

- The loops in a program may be concatenated.
- Two loops are concatenated to reach one after exiting the other, while still on a path from entry to exit.
- If the two loops are not on the same path, then they are not concatenated.



Concatenated loops

Unstructured loops

- This type of loops is really impractical to test and they must be redesigned or at least converted into simple or concatenated loops.

2.6 Data Flow Testing

- Data Flow Testing is a white-box testing technique that can be used to detect improper use of data values due to coding errors.
- Errors may be unintentionally introduced in a program by programmers.
- For instance:
 - A programmer might use a variable without defining it.
 - He may define a variable but not initialize it.

Example: `int a;`
 `if (a==67) { }`

- In this way, data flow testing gives a chance to look out of inappropriate data definition, its use in predicates, computations and termination.
- It defines potential bugs by examining the patterns in which that a piece of data is used.
- For example, if an out-of-scope data is being used in a computation, then it is a bug.

2.6.1 State of a Data Object

A data object can be in the following state:

Defined (d):

A data object is called defined when it is initialized.

Killed / Undefined / Released (k):

When the data has been reinitialized or the scope of a loop control variable is finished.

Usage (u):

- When the data object is on the right side of assignment or used as a control variable in a loop
- In general, we say that the usage is either **computational use** (c-use) or **predicate use** (p-use).

2.6.2 Data-Flow Anomalies

- Data flow anomalies represent the patterns of data usage which may lead to an incorrect execution of the

Anomaly	Explanation	Effect of Anomaly
du	Define-use	Allowed. Normal case.
dk	Define-kill	Potential bug. Data is killed without use after definition.
ud	Use-define	Data is used and then redefined. Allowed. Usually not a bug because the language permits reassignment at almost any time.
uk	Use-kill	Allowed. Normal situation.
ku	Kill-use	Serious bug because the data is used after being killed.
kd	Kill-define	Data is killed and then redefined. Allowed.
dd	Define-define	Redefining a variable without using it. Harmless bug, but not allowed.
uu	Use-use	Allowed. Normal case.
kk	Kill-kill	Harmless bug, but not allowed.

code.

- An anomaly is denoted by a two character sequence of actions.
- In addition to the above two character data anomalies, there may be single character data anomalies.
- To represent these types of anomalies, we take the following conventions:
 - ~x: indicates all prior actions are not of interest to x.
 - x~: indicates all post actions are not of interest to x.
- All single-character data anomalies are listed below:

Anomaly	Explanation	Effect of Anomaly
~d	First definition	Normal situation. Allowed.
~u	First Use	Data is used without defining it. Potential bug.
~k	First Kill	Data is killed before defining it. Potential bug.
D~	Define last	Potential bug.
U~	Use last	Normal case. Allowed.
K~	Kill last	Normal case. Allowed.

2.6.3 Terminology used in Data Flow Testing:

Definition Node

- Defining a variable means assigning value to a variable for the very first time in a program.
- Input statements, Assignment statements, Loop control statements, Procedure calls, etc.

Usage Node:

- It means the variable has been used in some statement of the program.
- Output statements, Assignment statements (Right), Conditional statements, Loop control statements, etc.
- A usage node can be of the following two types:
 - Predicate Usage Node: If usage node n is a predicate node, then n is a predicate usage node.
 - Computation Usage Node: If usage node n corresponds to a computation statement in a program other than predicate, then it is called a computation usage node.

Loop Free Path Segment

- It is a path segment for which every node is visited once at most.

Simple Path Segment

- It is a path segment in which at most one node is visited twice.
- A simple path segment is either loop-free or if there is a loop, only one node is involved.

Definition-Use Path (du-path)

- A du-path with respect to a variable v is a path between definition node and usage node of that variable.
- Usage node can be p-usage or c-usage node.

Definition-Clear path(dc-path):

- A dc-path with respect to a variable v is a path between definition node and usage node such that no other node in the path is a defining node of variable v .

2.6.4 Static Data Flow Testing

- Static analysis, the source code is analyzed without executing it. Let us consider an example

Example

Consider the program given below for calculating the gross salary of an employee in an organization. If his basic salary is less than Rs 1500, then HRA = 10% of basic salary and DA = 90% of the basic. If his salary is either equal to or above Rs 1500, then HRA = Rs 500 and DA = 98% of the basic salary. Calculate his gross salary.

```

main()
{
1.   float bs, gs, da, hra = 0;
2.   printf("Enter basic salary");
3.   scanf("%f", &bs);
4.   if(bs < 1500)
5.   {
6.       hra = bs * 10/100;
7.       da = bs * 90/100;
8.   }
9.   else
10.  {
11.      hra = 500;
12.      da = bs * 98/100;
13.  }
14.  gs = bs + hra + da;
15.  printf("Gross Salary = Rs. %f", gs);
16. }

```

Find out the define-use-kill patterns for all the variables in the source code of this application.

Solution

For variable 'bs', the define-use-kill patterns are given below.

Pattern	Line Number	Explanation
~d	3	Normal case. Allowed
du	3-4	Normal case. Allowed
uu	4-6, 6-7, 7-12, 12-14	Normal case. Allowed
uk	14-16	Normal case. Allowed
K~	16	Normal case. Allowed

For variable 'gs', the define-use-kill patterns are given below.

Pattern	Line Number	Explanation
~d	14	Normal case. Allowed
du	14-15	Normal case. Allowed
uk	15-16	Normal case. Allowed
K~	16	Normal case. Allowed

For variable 'da', the define-use-kill patterns are given below.

Pattern	Line Number	Explanation
~d	7	Normal case. Allowed
du	7-14	Normal case. Allowed
uk	14-16	Normal case. Allowed
K~	16	Normal case. Allowed

For variable 'hra', the define-use-kill patterns are given below.

Pattern	Line Number	Explanation
~d	1	Normal case. Allowed
dd	1-6 or 1-11	Double definition. Not allowed. Harmless bug.
du	6-14 or 11-14	Normal case. Allowed
uk	14-16	Normal case. Allowed
K~	16	Normal case. Allowed

2.6.5 Dynamic Data Flow Testing

- Dynamic data flow testing is performed with the intention to uncover possible bugs in data usage during execution of the code.
- Various strategies are employed for the creation of test cases.
- All these test strategies are defined below:

All-du Paths (ADUP)

- It states that every du-path from every definition of every variable to every use of that definition should be exercised under some test.
- It is the strongest data flow testing strategy; it is a superset of all other data flow testing strategies.

All-uses (AU)

- This states that for every use of the variable, there is a path from the definition of that variable to the use.

All-p-uses/Some-c-uses (APU + C)

- This strategy states that for every variable and every definition of that variable, include at least one dc- path from the definition to every predicate use.

All-c-uses / Some-p-uses (ACU + P):

- This strategy states that for every variable and every definition of that variable, include at least one dc- path from the definition to every computational use.

All-Predicate-Uses (APU):

- It is derived from the APU+C strategy and states that for every variable there is a path from every definition to every p-use of that definition.

All-Computational-Uses (ACU):

- It is derived from the ACU+P strategy and states that for every variable there is a path from every definition to every c-use of that definition.

All-Definition (AD):

- It states that every definition of every variable should be covered by at least one use of that variable.

Example

Consider the program given below. Draw its control flow graph and data flow graph for each variable used in the program, and derive data flow testing paths with all the strategies discussed above.

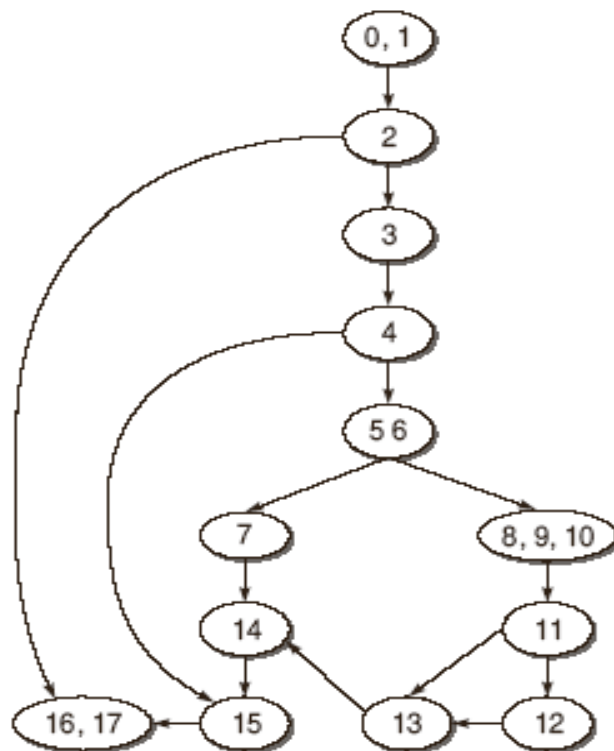
```

main()
{
    int work;
0.   double payment = 0;
1.   scanf("%d", work);
2.   if (work > 0) {
3.       payment = 40;
4.       if (work > 20)
5.       {
6.           if(work <= 30)
7.               payment = payment + (work - 25) * 0.5;
8.           else
9.           {
10.              payment = payment + 50 + (work - 30) * 0.1;
11.              if (payment >= 3000)
12.                  payment = payment * 0.9;
13.          }
14.      }
15.  }
16.  printf("Final payment", payment);

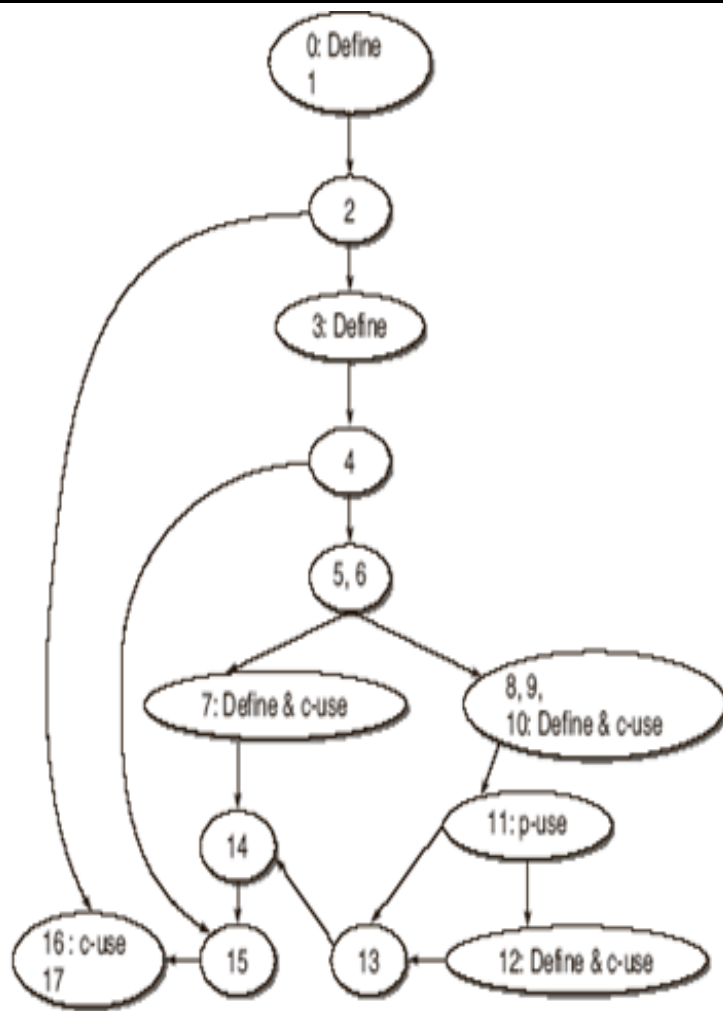
```

Solution

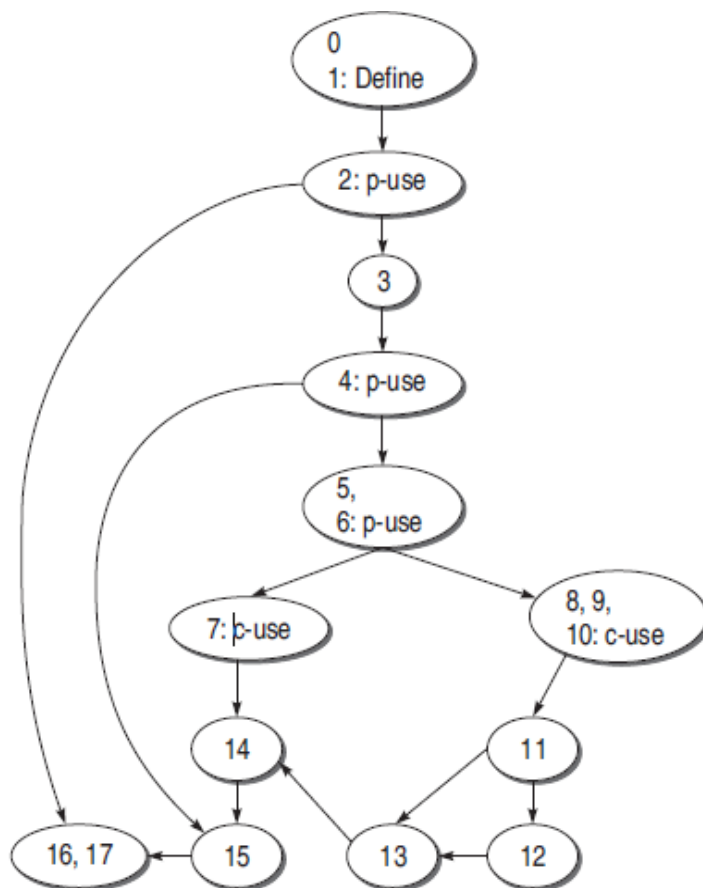
Control flow graph



The data flow graph for the variable 'payment'.



The data flow graph for the variable 'work'.



List of all the definition nodes and usage nodes for all the variables in the program.

Variable	Defined At	Used At
Payment	0,3,7,10,12	7,10,11,12,16
Work	1	2,4,6,7,10

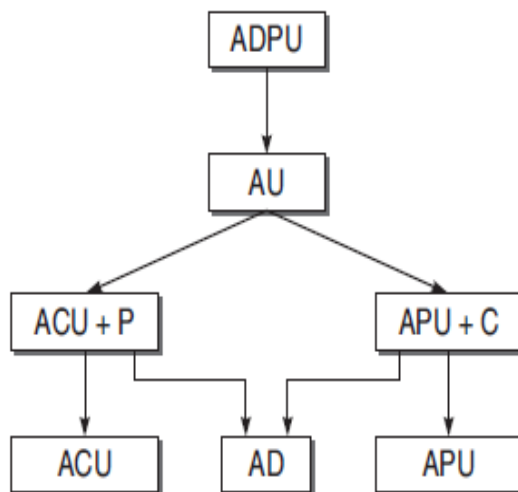
Data flow testing paths for each variable are:

Strategy	Payment	Work
All Uses(AU)	3-4-5-6-7 10-11 10-11-12 12-13-14-15-16 3-4-5-6-8-9-10	1-2 1-2-3-4 1-2-3-4-5-6 1-2-3-4-5-6-7 1-2-3-4-5-6-8-9-10
All p-uses (APU)	0-1-2-3-4-5-6-8-9-10-11	1-2 1-2-3-4 1-2-3-4-5-6

All c-uses (ACU)	0-1-2-16 3-4-5-6-7 3-4-5-6-8-9-10 3-4-15-16 7-14-15-16 10-11-12 10-11-13-14-15-16 12-13-14-15-16	1-2-3-4-5-6-7 1-2-3-4-5-6-8-9-10
All-p-uses / Some-c-uses (APU + C)	0-1-2-3-4-5-6-8-9-10-11 10-11-12 12-13-14-15-16	1-2 1-2-3-4 1-2-3-4-5-6 1-2-3-4-5-6-8-9-10
All-c-uses / Some-p-uses (ACU + P)	0-1-2-16 3-4-5-6-7 3-4-5-6-8-9-10 3-4-15-16 7-14-15-16 10-11-12 10-11-13-14-15-16 12-13-14-15-16 0-1-2-3-4-5-6-8-9-10-11	1-2-3-4-5-6-7 1-2-3-4-5-6-8-9-10 1-2-3-4-5-6
All-du-paths (ADUP)	0-1-2-3-4-5-6-8-9-10-11 0-1-2-16 3-4-5-6-7 3-4-5-6-8-9-10 3-4-15-16 7-14-15-16 10-11-12 10-11-13-14-15-16 12-13-14-15-16	1-2 1-2-3-4 1-2-3-4-5-6 1-2-3-4-5-6-7 1-2-3-4-5-6-8-9-10
All Definitions (AD)	0-1-2-16 3-4-5-6-7 7-14-15-16 10-11 12-13-14-15-16	1-2

2.6.6 Ordering of Data Flow Testing Strategies

- The following is the path-flow and data-flow testing strategies.
- The arrows denote that the strategy at the arrow's tail is stronger than the strategy at the arrow's head



2.7 Mutation Testing

- Mutation testing is the process of mutating some segment of code (putting some error in the code) and then testing this mutated code with some test data.
- If the test data is able to detect the mutations in the code, then the test data is quite good, otherwise we must focus on the quality of test data.
- During mutation testing, faults are introduced into a program by creating many versions of the program, each of which contains one fault.
- Test data are used to execute these faulty programs with the goal of causing each faulty program to fail.
- Faulty programs are called *mutants* of the original program and a mutant is said to be *killed* when a test case causes it to fail.
- When this happens, the mutant is considered *dead* and no longer needs to remain in the testing process.

2.7.1 Primary Mutants

- When the mutants are single modifications of the initial program using some operators they are called **primary mutants**.
- Mutation operators are dependent on programming languages.

Example: if(a>b)
 x = x+y;
 else
 x=x-y;
 printf(“%d”,x);

The following mutants for the above example:

M1: x = x - y;
M2: x = x / y;
M3: x = x + 1;
M4: printf(“%d”, y);

The results of the initial program and its mutants are shown below:

Test Data	x	y	Initial Program Result	Mutant Result
TD1	2	2	4	0 (M1)
TD2(x and y # 0)	4	3	7	1.4 (M2)
TD3 (y #1)	3	2	5	4 (M3)
TD4(y #0)	5	2	7	2 (M4)

2.7.2 Secondary Mutants

- Multiple levels of mutation are applied on the initial program is called secondary mutants.
- In this case, it is very difficult to identify the initial program from its mutants.

Example

```

if (a < b)
    c = a;

```

Mutants for this code may be as follows:

```

M1: if (a <= b-1)
    c = a;
M2: if (a+1 <= b)
    c = a;
M3: if (a == b)
    c = a+1;

```

Example

Consider the program *P* shown below.

```

r = 1;
for (i = 2; i<=3; ++i)
{
    if (a[i] > a[r])
        r = i;
}

```

The mutants for *P* are:

```

M1:  r = 1;
      for (i = 1; i<=3; ++i)
      {
          if (a[i] > a[r])
              r = i;
      }
M2:  r = 1;
      for (i = 2; i<=3; ++i)
      {
          if (i > a[r])
              r = i;
      }

```



```

M3:  r = 1;
      for (i = 2; i <= 3; ++i)
      {
          if (a[i] >= a[r])
              r = i;
      }
M4:  r = 1;
      for (i = 1; i <= 3; ++i)
      {
          if (a[r] > a[i])
              r = i;
      }

```

Consider the following test data selection:

	a[1]	a[2]	a[3]
TD1	1	2	3
TD2	1	2	1
TD3	3	1	2

Apply these test data to mutants, M1, M2, M3, and M4.

	P	M1	M2	M3	M4	Killed Mutants
TD1	3	3	3	3	1	M4
TD2	2	2	3	2	1	M2 and M4
TD3	1	1	1	1	1	none

- M1 and M3 are not killed by the test data selection. Therefore, the test data is incomplete.
- Therefore, we need to add a new test data, TD4 = {2,2,1}, then this test data kills M3.

2.7.3 Mutation Testing Process

The mutation testing process steps as follows:

1. Construct the mutants of a test program.
2. Add test cases to the mutation system and check the output of the program on each test case to see if it is correct.
3. If the output is incorrect, a fault has been found and the program must be modified and the process restarted.
4. If the output is correct, that test case is executed against each mutant.
5. If the output of a mutant differs from that of the original program on the same test case, the mutant is assumed to be incorrect and is killed.
6. After each test case has been executed against each mutant, each remaining mutant falls into one of the following two categories.

- a. One, the mutant is functionally equivalent to the original program. That means produces the same output as the original program, so no test case can kill it.
- b. Two, the mutant is killable, but the set of test cases is insufficient to kill it. In this case, new test cases need to be created.
- c. The mutation score for a set of test data is the percentage of non-equivalent mutants killed by that data. If the mutation score is 100%, then the test data is called mutation-adequate.